Executing SQL Stored Procedures from inside a Web Application Part 2

By Don Schlichting

How to pass parameters (variables) IN, OUT, and RETURN error codes from a Stored Procedure to a Web Page.

Introduction

In the first article of this series, the reasons for using stored procedures as well as the relatively easy learning curve involved were discussed. In addition, an example involving a stored procedure being executed from inside a Dot Net web page was presented. The stored procedure executed a simple select with no variables and its results were bound to a Grid View. This article will continue with the same example but add a method to pass in variables (parameters) to control a WHERE clause.

Parameters

Parameters are objects that allow a calling application to pass variables to, or receive variables from, a SQL Stored Procedure. There are three types of SQL Parameters;

1. INPUT – An INPUT parameter is used to pass a value into a stored procedure. A common example would be a web page passing in values that will be used in the WHERE clause of a SQL Stored Procedure.

2. OUTPUT – OUTPUT parameters send a variable from a procedure back to the calling program. For example, if a procedure created a new customer in the database, we may want the CustomerID returned back to our application so it can pass it to the next web page.

3. RETURN - RETURN parameters are used to send the execution success or failure of the stored procedure back. Using the customer example above, if the application sends in a customer that already exists, the stored procedure could send back an INT error code indicating that a duplicate insert was attempted.

INPUT Parameters

In the example from the previous article, a stored procedure was executed from a web page and the results were bound to a Grid View. This example will expand on that code by passing a variable from the web page to a stored procedure's WHERE clause. The Adventure Works database will be used to return summed quantities and dollar amounts of sold products. Passing variables into a stored procedure uses a SQL parameter type called INPUT. Create a new stored procedure with the following statement:

```
USE AdventureWorks
GO

CREATE PROCEDURE testProc2
(
@CutOffAmt money
)
```

```
AS
SELECT OrderQty, LineTotal
FROM Sales.SalesOrderDetail
WHERE LineTotal >= @CutOffAmt;
```

INPUT parameters are declared using the @ symbol. This example uses the code segment "@CutOffAmt money". The @ symbol is required prior to the variable name. It instructs SQL this new variable will be local. So even if two users simultaneously execute this procedure, they will each have their own @CutOffAmt. The variable is destroyed when the SQL statement ends. Notice that unlike other variables, the INPUT paramter does not require the SQL key word DECLARE. In addition, the INPUT parameter is defined above the AS key word, unlike usual local variables. Both INPUT and OUTPUT paramaters must be defined above the AS key word. In addition, the paramters must be defined between an opening and closing parenthasis. The last line of the statement uses the paramater in its WHERE clause.

As displayed in the image below, the web page example will accept an amount from a text box to be used as the parameter value. The value will limit the number of rows returned to those greater than the amount passed in the text box. The results will be bound to a Grid View.

The web page itself is straight forward, containing a text box, button, and grid.

```
<html xmlns="http://www.w3.org/1999/xhtml" >
<head id="Head1" runat="server">
<title>Test 2</title>
</head>
<body>
<form id="form1" runat="server">
<div>

Enter a cut off amount:
<p><asp:TextBox ID="tbAmount" runat="server"></asp:TextBox></p>
<p><asp:Button ID="btnExecute" runat="server" Text="Get Data"
OnClick="Execute_Clicked" /></p>
```

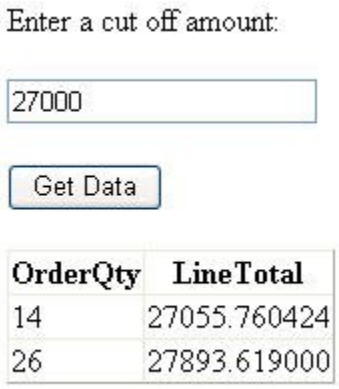
```
<p><asp:GridView ID="GridView1" runat="server"></asp:GridView></p>

</div>
</form>
</body>
</html>
```

Prior to the html code, enter the following script statements. These will control the button click which in turn will execute the stored procedure.

```
 <%@ Page Language="C#" %>
<script runat="server">
protected void Execute_Clicked(object sender, EventArgs e)
{
System.Data.SqlClient.SqlConnection objConn = new
System.Data.SqlClient.SqlConnection();
objConn.ConnectionString = "server=.; database=AdventureWorks; uid=sa; pwd=test;";
objConn.Open();
System.Data.SqlClient.SqlCommand objCmd = new
System.Data.SqlClient.SqlCommand("testProc2", objConn);
objCmd.CommandType = System.Data.CommandType.StoredProcedure;
System.Data.SqlClient.SqlParameter pCutOffAmt = objCmd.Parameters.Add("@CutOffAmt",
System.Data.SqlDbType.Int);
pCutOffAmt.Direction = System.Data.ParameterDirection.Input;
pCutOffAmt.Value = Convert.ToInt16(tbAmount.Text);
GridView1.DataSource = objCmd.ExecuteReader();
GridView1.DataBind();
objConn.Close();
}
</script>
```

Running the web page with an amount of 27,000 entered as a cutoff, two rows will be returned. Demonstrating that the value in the text box was used as the WHERE amount in the stored procedure.



OrderQty	LineTotal
14	27055.760424
26	27893.619000

Like all things Dot Net, there are several different methods of executing stored procedures. All the examples in this series will use one method. This one method will accommodate

INPUT, OUTPUT, and RETURN parameters, or no parameters at all (as exampled in part 1). There are three lines of code for the parameter.

```
System.Data.SqlClient.SqlParameter
pCutOffAmt = objCmd.Parameters.Add("@CutOffAmt",
System.Data.SqlDbType.Int);
```

In this first line, the Dot Net SqlParameter is given a parameter name of "pCutOffAmt". This is the name Dot Net will use to reference the paramter. On the other half of the equasion, pCutOffAmt is equated to the SQL paramter @CutOffAmt. Notice that the Dot Net name and the SQL name do not have to be the same. However, the objCmd.Parameter name of @CutOffAmt must match the parameter name inside the stored procedure. The last part of the equasion tells Dot Net what type of SQL data type the parmater is, "Int" in this example. Again this must match the data type in the SQL stored procedure.

In the next code line:

```
pCutOffAmt.Direction =
System.Data.ParameterDirection.Input;
```

Dot Net is told the direction of the paramter, either Input, Output, or Return. By default, inside a SQL stored procedure, a parmater is INPUT if its not explictily makred as OUTPUT. In our stored procedure example, *(@CutOffAmt money),* is not marked as OUTPUT, therefore it is by default an INPUT.

The last Dot Net parameter line:

```
pCutOffAmt.Value = Convert.ToInt16(tbAmount.Text);
```

This sets the value of the parameter. In this example, the value is coming from the amount entered in the text box. It could have just as easily have been a fixed amount or passed from a proceeding web page.

Conclusion

This example demonstrated the ease of which a value can be passed from a web page into a stored procedure and used in the procedures WHERE clause. There have been multiple requests for the instructions on how to use stored procedures to return the value of an AUTO INDENTITY field to a web page. So next month, we'll continue into OUTPUT parameters.